UNIVERSITY OF BOLOGNA, CESENA CAMPUS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

February 3, 2026

Assignment #03 — Smart Tank Monitoring System

Embedded Systems and Internet of Things Course

by

Nicholas Magi 0001113915
Rebecca Scarcelli 0001118585
Elettra Ventura 0001117272

Abstract: The project requires a 4-tier IoT system to monitor rainwater levels and control a drainage valve. A PC back-end coordinates an ESP32 sensor, Arduino valve controller, and web dashboard via MQTT, HTTP, and Serial. It automates drainage based on specific water thresholds but allows local or remote manual overrides. Deliverables include FSM-based code for all subsystems and a technical report with schematics and a video demo.

Contents

1 Introduction **3**

1.1 Requirements — v1.0.0-20251112 3

2 System Architecture - Control Unit Subsystem (CUS) **8**

2.1 Implementation 9

3 Water Channel Subsystem - (WCS) **10**

3.1 Implementation 11

4 Dashboard Subsystem - (DBS) **12**

4.1 Implementation 13

5 Tank Monitoring Subsystem - (TMS) **13**

5.1 Implementation 14

1 Introduction

1.1 Requirements — v1.0.0-20251112

We want to realise an IoT system monitoring the rainwater level of a tank. The system is composed of 4 subsystems:

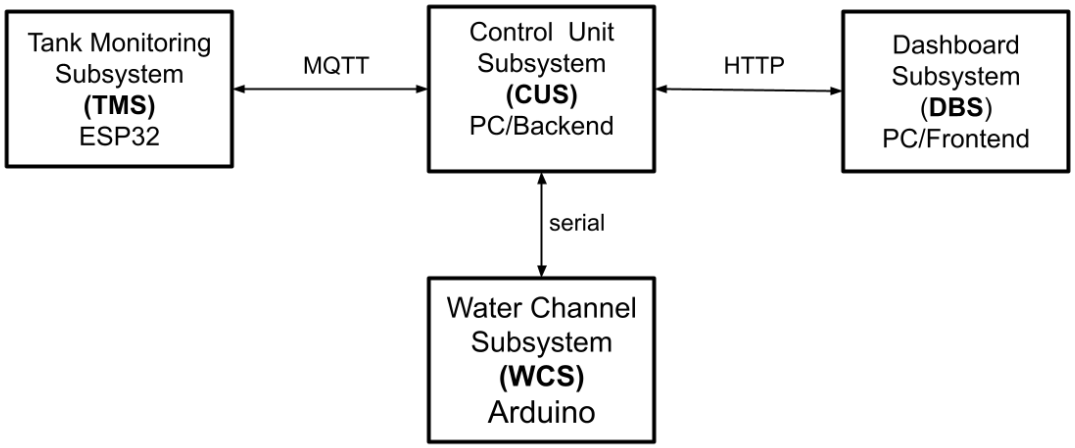

Figure 1: System's simplified architecture diagram

Description

Components The system is composed by four subsystems:

- **Tank Monitoring subsystem (TMS)** - based on ESP
  - embedded system to monitor the rainwater level in the tank
  - It interacts with the Control Room subsystem (CUS) via MQTT
- **Control Unit subsystem (CUS)** - Back-end, running on a PC
  - main subsystem, governing and coordinating the whole system
  - It interacts via MQTT with the Tank Monitoring Subsystem (TMS)
  - It interacts through the serial line with the Water Channels Subsystem

(WCS)

  - It interacts via HTTP with the Dashboard Subsystem (DBS)

- **Water Channel Subsystem (WCS)** - based on Arduino
  - Embedded system controlling the water channel connecting the tank with the water channel network.
  - It interacts via serial line with the Control Unit Subsystem (CUS).
  - It provides a panel for human operators to interact in place.
- **Dashboard subsystem (DBS)** - Frontend/web app, running on the PC or any device
  - Front-end for remote operators to visualise data and interact with the system.
  - it interacts via HTTP with the Control Unit Subsystem.

Hardware components

- **TMS**
  - SoC ESP32 board (or ESP8266) including:
  - 1 sonar
  - 1 green led
  - 1 red led
- **WCS**
  - Microcontroller Arduino UNO board including:
  - 1 servo motor
  - 1 potentiometer

  - 1 tactile button
  - 1 LCD display

Description of the behaviour The system is meant to monitor the rainwater level inside the tank, and - depending on values - controlling the opening of a water channel connecting the tank to a network of water channers. The overall system can be in two different modes: `AUTOMATIC` or `MANUAL`. In `AUTOMATIC` mode, the system automatically controls the opening of the water channel, depending on the current rainwater level in the tank. In `MANUAL` mode, the opening is controlled manually by an operator. The starting mode when booting is `AUTOMATIC`.

Details

- The **TMS** is responsible for continuously monitoring the level of the rainwater level, by means of proper sensors (e.g. a sonar).
  - The rainwater level are sampled at frequency F and sent to the **CUS** subsystem.
  - When the system is working correctly (network ok, sending data ok) the green led is on and the red is off; otherwise – in the case of network problems – the red led should be on and the green led off.
- The **WCS** subsystem is responsible for controlling a valve (with a motor) opening/closing a water channel draining water from the tank to the water channel network.
  - The opening range is in percentage: from 0% = channel closed (motor position 0 degree), up to 100% = channel full open (motor position 90° degree).
  - The water channel opening level depends on the state of the system, established by the **CUS** subsystem (see later).
  - The **WCS** includes a button to enable the `MANUAL` mode, in particular:
    * When the button is pressed, the system enters in `MANUAL` mode, so that the water channel opening level can be manually controlled by operators using a potentiometer.

    - To exit from the `MANUAL` model, the button must be pressed again.
  - The **WCS** subsystem is equipped also with an LCD display reporting:
    - the current valve opening level of the water channel.
    - the current mode (`AUTOMATIC` or `MANUAL`), or `UNCONNECTED` (see later).
- The **CUS** subsystem is in charge of the policy governing the behaviour of the system. In particular:
  - Rainwater level monitoring
    - When the rainwater level exceeds the level L_1 (but below L_2, with $L_1 < L_2$) for more than T_1 time, the water channel is opened at 50% until the rainwater level is below L_1.
    - If the the rainwater level exceeds the level L_2, the water channel is immediately opened at 100%, until the value is below L_2.

If the **CUS** is not receiving data for more than T_2 time units from the **TMS** (because of network problems), then the system enters into an `UNCONNECTED` state, which is restored to a normal state only if/when the network problems are solved.

- The **DBS** subsystem is a dashboard for visualising the state of the Tank Monitoring system, including:
  - A graph of the rainwater level, considering the last N measurements.
  - The current value of the valve opening percentage.
  - The state of the system: `MANUAL`, `AUTOMATIC`, `UNCONNECTED` or `NOT AVAILABLE`.
    - the state is labelled as `NOT AVAILABLE` when the **CUS** is not reachable, for any reason.

Besides, it includes:

- a GUI button to switch the modality (`MANUAL`, `AUTOMATIC`).
- a GUI widget to control the opening level of the valve in the **WCS** (when in

`MANUAL` mode).

The assignment

Design and develop a prototype of the Tank Monitoring system, considering as further requirements:

- About the **TMS** subsystem:
  - it must run on an ESP32 (or an equivalent SoC board).
  - The control logic must be designed and implemented using finite state machines (synchronous or asynchronous) and (possibly, if useful) task-based architectures, exploiting the RTOS.
  - Must use MQTT to communicate with the **CUS** subsystem.
- About the **WCS** subsystem:
  - It must run on an Arduino (or an equivalent MCU board).
  - The control logic must be designed and implemented using finite state machines (synchronous or asynchronous) and (possibly, if useful) task-based architectures.
  - It must communicate with the **CUS** subsystem via serial line.
- About the **CUS** subsystem:
  - It must run on a PC, as a server/back-end.
  - No specific constraints about the programming/sw technology to be used.
  - It must use:
    - MQTT to communicate with the **TMS** subsystem.
    - HTTP to communicate with the **DBS** subsystem.
    - the serial line to interact with the **WCS**.

- About the **DBS** subsystem
  - It must run on a PC
  - No specific constraints about the programming/sw technology to be used.
  - It must use HTTP to interact with the **CUS** subsystem.

For any aspect not specified, you are free to choose the approach you consider more useful or valuable.

The Deliverable

The deliverable consists in a zipped folder assignment-03.zip including 4 subdirectories - one for each subsystem: `tms`,`cus`, `wcs`,`DBS`) - and a `doc` subdirectory, including the report. The report must be in PDF or MD (`report.pdf` or `report.md`) must include a high-level description of the behaviour of subsystems (using FSMs), a representation of the schema/breadboard and the link to a short video demonstrating the system.

2 System Architecture - Control Unit Subsystem (CUS)

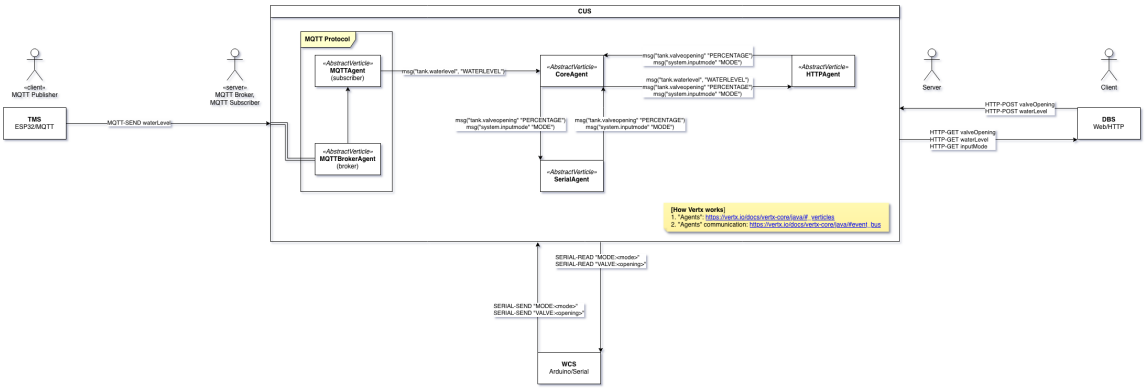

Figure 2: System architecture

The core of the whole system is the CUS: **Control Unit Subsystem**. It is the nevralgic center of the communication, and contains the logic responsible in open-

ing and closing the valve when the system is in `AUTOMATIC` mode and the water levels reach certain thresholds.

2.1 Implementation

The system is written in Java, and the `vertx` library has been used to implement internally all the **agents** (`verticles`) needed to interact with all the different protocols involved.

In particular, we have:

1. `CoreAgent`: contains the previously described logic, implemented using some timers;
2. `HttpAgent`: acts like a small HTTP server and provides all the API endpoints required by the DBS to send/receive the data;
3. `SerialAgent`: interface to WCS; it manages serial communication;
4. `MqttBroker`: serves like an embedded MQTT broker that manages client connections and handles the data stream from the TMS via the vertx-mqtt library;
5. `MqttAgent`: the MQTT client that is subscribed to `tank/waterlevel` topic and constantly receives sampled values from TMS.

3 Water Channel Subsystem - (WCS)

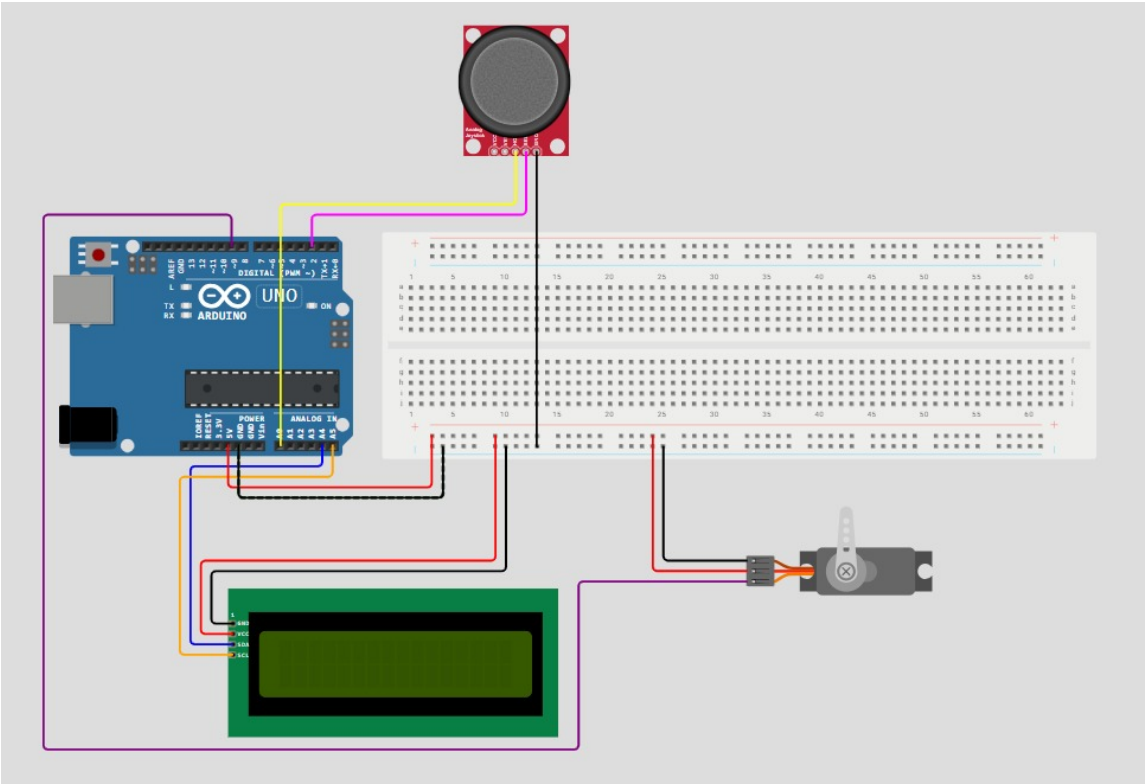

Figure 3: Subsystem circuit.

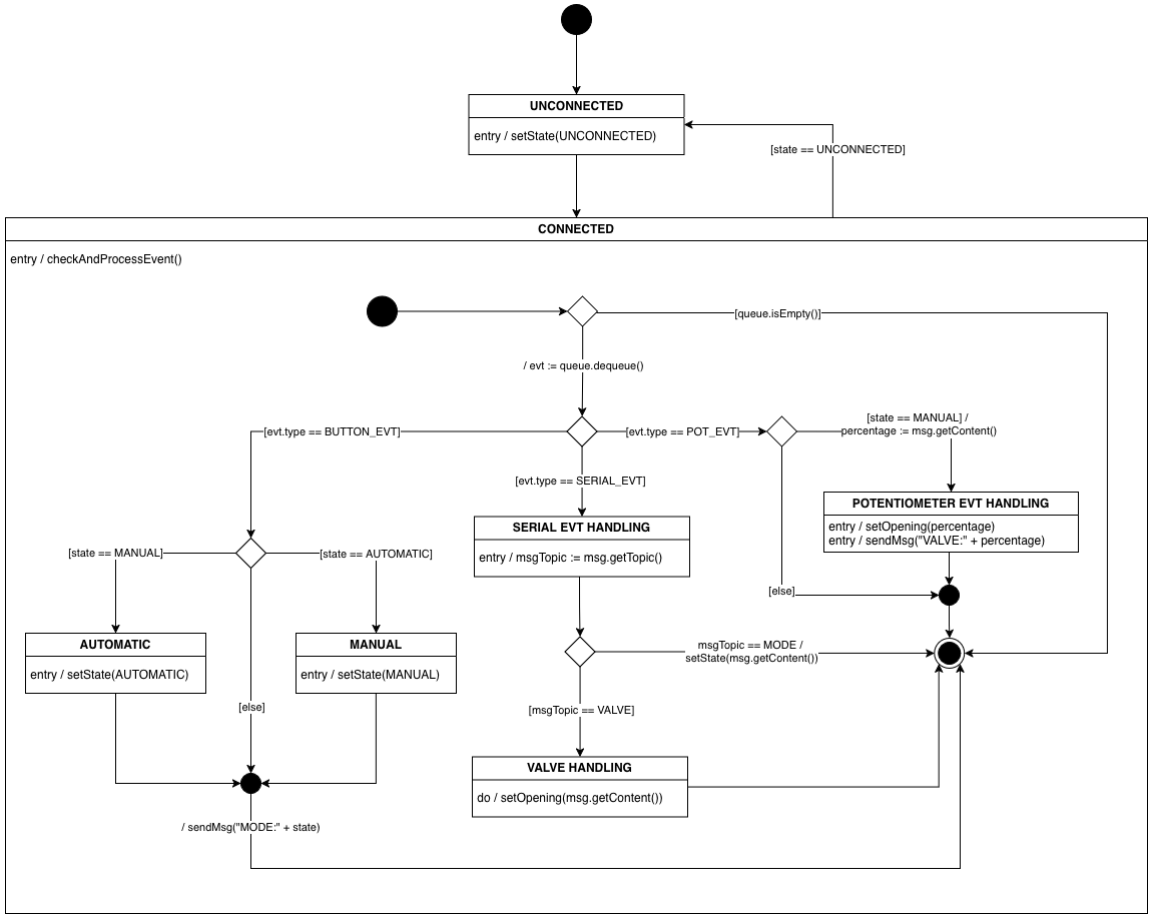

Figure 4: Subsystem asynchronous FSM diagram

3.1 Implementation

The Water Channel Subsystem's behaviour, by its definition from the requirements, is driven by many events: as described by the Fig. 4, we identified three types of them:

- `ButtonEvent` happens via interrupts when the button is clicked, and it can toggle the system state to either `AUTOMATIC` or `MANUAL`;
- `SerialEvent` is fired whenever a message is received from the Serial bus: messages regarding the input mode of the system or the opening of the

valve;

- `PotentiometerEvent` tells that the potentiometer acting as a lever has changed, so the valve should open accordingly (if the system is in `MANUAL` mode);

The events processing is handled by the class `AsyncFSM`, which reads events via **polling** from the shared **event queue** and acts consequently.

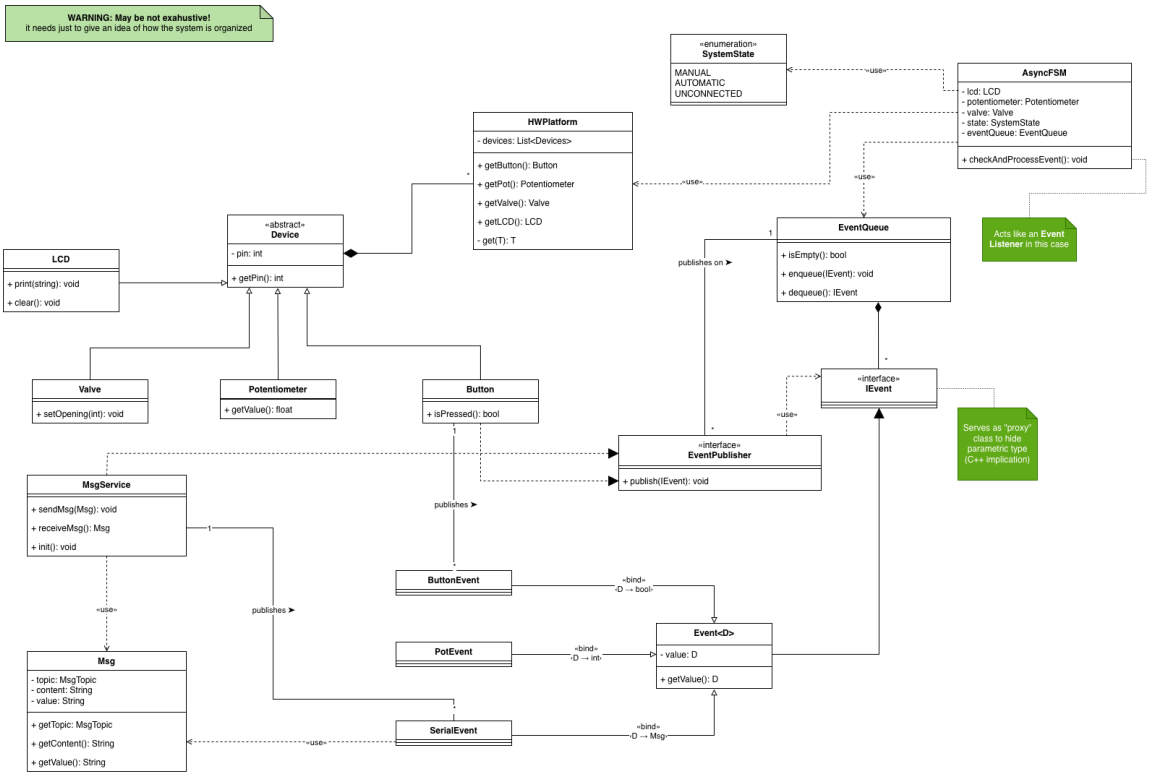

Figure 5: Subsystem class diagram (may not be exhaustive!)

4 Dashboard Subsystem - (DBS)

The **Dashboard Subsystem (DBS)** represents the graphical interface of the project, designed to provide human operators with an overview of the state of the system and remote control capabilities. It communicates exclusively with the **Control Unit Subsystem (CUS)** via asynchronous HTTP requests. It includea a visual mapping of raw JSON data into a level chart and it manages the operational modes.

4.1 Implementation

The subsystem is implemented using standard web technologies: **HTML5**, **CSS** for a responsive and intuitive layout, and **JavaScript** for dynamic data fetching from the CUS' APIs. It performs periodic polling of the "status" endpoint, allowing the interface to reflect the current state of the tank and the valve without requiring a page refresh. This ensures that the operator is always presented with a real-time "digital twin" of the physical system.

5 Tank Monitoring Subsystem - (TMS)

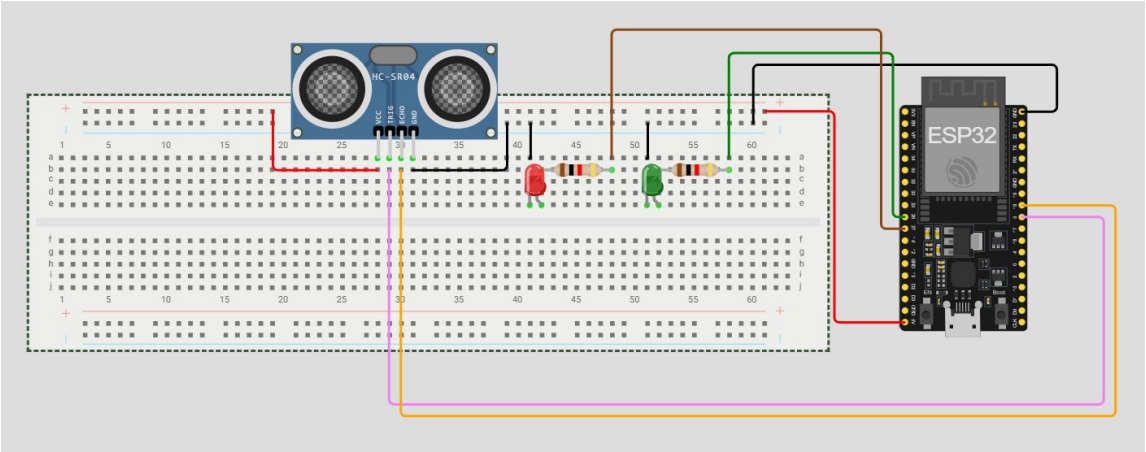

Figure 6: Subsystem circuit.

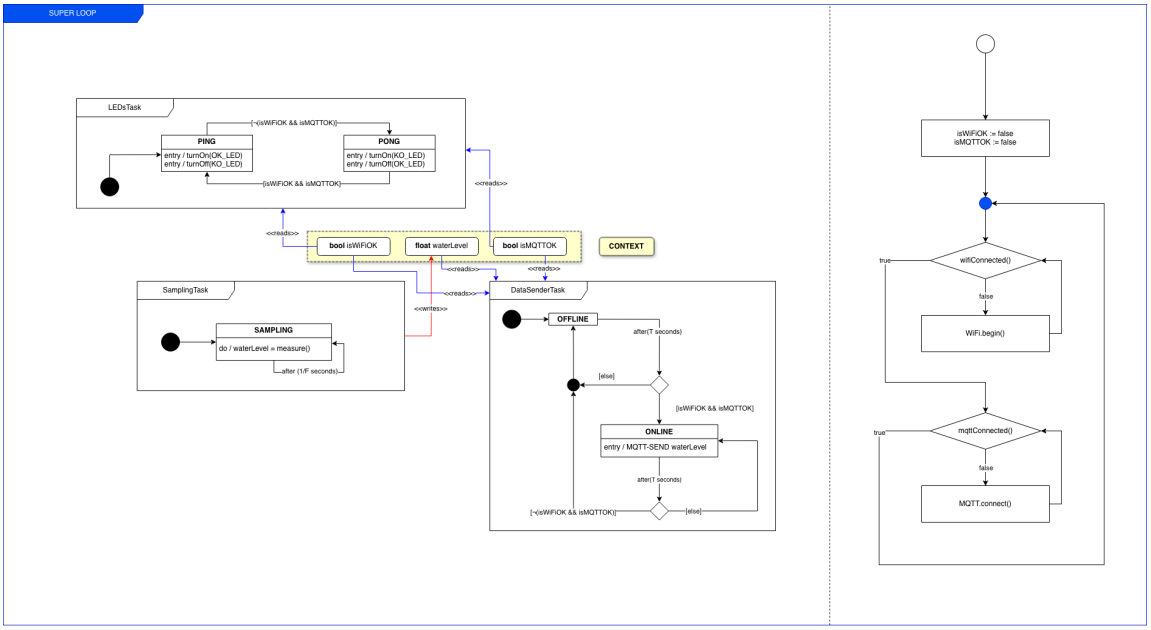

Figure 7: Subsystem synchronous FSM.

5.1 Implementation

The system requirements talk about data sampling carried out periodically, so we immediately thought that designing the system with a **Synchronous Finite State Machine** would carry out the task in the best way possible. We designed 3 main tasks, each briefly described as follows:

- `LEDsTask` is the task responsible for handling the LEDs according to the requirements - the KO Led is turned on when the system is disconnected (i.e. cannot reach MQTT Broker), otherwise is turned off (and the reverted logic applies to the OK Led);

- `SamplingTask` constantly reads the sampled values from the Sonar sensor and stores them into a shared variable;

- `DataSenderTask` reads the sampled values received from the `SamplingTask` and, if there's an estabilished connection, sends them via MQTT to the broker.

The connection of the whole subsystem is managed directly inside the superloop because we thought it as an asynchronous operation: at any point in time disconnection can happen, and therefore the system status must be updated almost in real-time.